

PROJECT_CONTEXT

Purpose

Build a lightweight personal “send pipe” to transfer files, text, and clipboard between a user’s own devices over peer-to-peer WebRTC DataChannels, using only a minimal signaling server for discovery and connection setup.

Architecture Overview

- `server/`: WebSocket signaling server (presence, pairing sessions, SDP/ICE relay)
- `app/`: Desktop app (Tauri shell + Vite/React UI), identity, pairing, WebRTC, transfers
- `shared/`: Shared protocol types + small helpers (safety phrase, chunk framing)

Current Phase

App shell started: Tauri wrapper around the UI with native save-to-Downloads for received files.

Account mode started: Google device-flow login on the signaling server; WS presence is scoped per user.

LAN discovery started: server responds to UDP discovery; app auto-finds the local signaling server in Tauri.

Dev convenience: Tauri can auto-start the Node signaling server if none is found on LAN (spawns `npm run dev -w @landrop/server`).

Auth gating: when LAN server requires auth, the app delays opening the WebSocket until after sign-in.

Transfer handshake hardened: sender waits for explicit receiver acceptance before chunking; cancel/timeout paths are implemented on both sides.

Session lifecycle improved: local sign-in/sign-out now updates auth state without full window reload; auth-required/no-token state clears active peer session and presence.

Connection management expanded: per-device connect/disconnect actions in Presence, plus Saved Connections with per-device forget.

UI refreshed: moved to a cleaner modern visual system (glass cards, styled controls, simplified hierarchy, subtle motion transitions).

Device connect UX refreshed: replaced the connect dropdown with a live device grid and platform icons (Windows/macOS/Linux/Android/iOS/Web) derived from detected device type.

UI density pass: removed redundant signaling/presence clutter, tightened card spacing, standardized control sizes, and replaced the checkbox-style findable control with a modern toggle button.

Discovery UX clarified: "Saved Connections" was renamed/scoped to "Trusted Devices (Pairing)" (non-account mode), while account mode now emphasizes "Available Devices" with same-account + findable guidance.

Information architecture updated: app now separates discovery into "My Devices" (same-account LAN discovery) and "Other Users" (pairing/trusted users + incoming transfer requests), reducing ambiguity and text clutter.

Compact app pass: reduced top/bottom whitespace, tightened card/control spacing, simplified the header, and lowered default Tauri window size to better fit a focused file-transfer workflow.

Share flow updated: same-account devices are visible without pairing codes; cross-account sharing now uses request/approve semantics (`Other Users`) with `Findable` gating instead of code entry.

Layout updated: account+device controls are a compact top strip, while `My Devices` and `Other Users` are side-by-side scrollable panels; outer app surface remains non-scrollable.

UI simplification pass: reduced instructional text, replaced repeated labels with icon/state chips, and kept interactions action-first for fast file-sharing use

Top-bar polish: reworked the compact strip into grouped controls (identity/device/findable vs auth/actions), condensed auth inputs into a secondary row, and turned connection errors into a compact alert chip.

Top controls simplified further: removed redundant signed-in/disconnect status pills and switched to single stateful action buttons (`Sign in/Sign out`, `Disconnect/Disconnected`).

Credential persistence adjusted: local email/password fields are now persisted in profile-scoped local storage and retained across sign-in/sign-out so new sessions can reuse the same credentials.

Panel layout lock: `My Devices` and `Other Users` remain side-by-side in desktop app layout (removed responsive single-column collapse).

Account matching fix: server now honors portable `local:\*` auth token scope even when strict auth mode is off, so same email/password devices correctly appear under `My Devices` after sign-in.

Guest visibility refinement: non-logged-in devices can be discoverable (`Findable`) and requestable, but are never treated as the same account scope (only explicit `local:\*` token matches count as same account).

Planned Next Phase (Local Account Matching MVP)

Goal: keep the UX “sign in once, see devices, send instantly” without introducing an external database service yet.

Product behavior (what user sees)

- Sign in with email + password on each device.
- After sign-in, app auto-connects to LAN signaling and shows only devices in the same account scope.
- “Findable” toggle controls discoverability per device (ON = visible/receivable, OFF = hidden/reject direct connect).
- Device list supports per-device actions:
 - Disconnect this device (drops active session for that target only).
 - Forget this device (remove remembered/trusted relation locally).
- Auto-login on app reopen using persisted local session.
- Sign out revokes local session and disconnects from signaling/peers.

Authentication model (MVP without external DB)

- Keep portable local account scope token derived from normalized `email:password`.
- Treat that derived token as the account key used for presence scoping and pairing authorization.
- Do not send raw password over WebSocket signaling; token only.
- Persist only session/token material locally (OS keychain target later; localStorage fallback for dev).

Security posture (explicit)

- This is a LAN-first convenience auth model, not production-grade identity.
- Anyone who knows the same email+password can impersonate that account on LAN.
- No recovery/reset/email verification/rate limiting in this phase.
- Next hardening phase should replace this with real server-verified auth + persistent user store.

Implementation phases

1. Session lifecycle and startup flow
 - App boot order: restore session -> discover LAN server -> connect WS -> publish presence.
 - Ensure reconnect path preserves account scope after temporary disconnects.
2. Device-state features
 - Finalize findable toggle behavior end-to-end (presence visibility + connecting).
 - Persist editable device name and apply immediately without full app restart
3. Connection management UX
 - Add per-device “disconnect” and “forget” actions in UI.
 - On local sign-out: close active peer/data channels, clear auth token, clear presence state.
4. Stability and observability

- Add structured logs for auth/session/connect/disconnect events.
 - Add guardrails for stale sessions and duplicate device instances.
5. Documentation and migration notes
- Document environment flags and exact behavior of portable account mode.
 - Define migration path to DB-backed auth (SQLite/Postgres + real signup/login).

Key Design Decisions

- WebSocket signaling is JSON messages with explicit `type` fields.
- Pairing uses short numeric codes and “safety phrase” verification derived from exchanged public keys.
- WebRTC uses STUN only; if connection fails, the UI reports it.
- DataChannel uses JSON control messages plus binary framed file chunks.
- File receive prefers native write to Downloads via a Tauri command; web download fallback remains for dev.
- Dev simulation supports multiple identities via URL `?profile=...` (namespaces browser storage keys).
- Optional account mode: server issues session tokens after Google device login; clients send `authToken` in `hello`; presence and signaling are scoped to the same `user.sub`.
- Local auth option for testing: email+password endpoints issue the same session token used by WS auth.
- Portable local auth option for LAN: clients derive `authToken = local:<sha256(email:password)>` so any server can scope the same account without shared DB/sessions.
- LAN discovery uses UDP broadcast to find a signaling server on the same network (Tauri only; web dev still uses env URLs).
- Device discoverability is explicit: `findable` is sent in `hello`, presence only includes findable devices, and direct signaling relay rejects non-findable targets.
- File transfer protocol now enforces `FILE\_OFFER -> ACCEPT/DECLINE -> chunk stream -> DONE/CANCEL` with explicit UI-visible phases.

File Structure

- `shared/src/protocol.ts`: WS + DataChannel message types
- `shared/src/safetyPhrase.ts`: safety phrase derivation
- `shared/src/chunkFraming.ts`: binary chunk header framing
- `server/src/index.ts`: signaling server
- `app/src/lib/\*`: client modules (identity, ws, webrtc, transfer)
- `app/src-tauri/src/commands.rs`: native commands (save to Downloads)
- `app/src-tauri/icons/icon.png`: placeholder app icon
- `app/src/lib/authClient.ts`: Google device-flow helper (server endpoints)
- `app/src/lib/discoveryClient.ts`: Tauri LAN signaling discovery helper

Known Issues / TODO

- No TURN; strict NATs may fail.
- Trust model is minimal (no signatures yet); pairing safety phrase is informational only.
- Clipboard sync is stubbed; requires loop prevention and per-OS integration for “desktop”.
- Integration tests not added yet.
- App device-name edits must not restart signaling; name changes now only send a new `hello` message.
- Running “two devices” in the same browser window requires different `?profile=` values; otherwise they share a `deviceId` and will overwrite each other server-side.
- Tauri permissions/capabilities are not locked down yet (needs an explicit allowlist later).
- Tauri scripts run via `npx tauri ...` (no global `tauri` required).
- Auth uses Google `tokeninfo` (simple MVP verification). Replace with JWKS verification (or a proper auth service) before shipping.
- Local auth is for testing only (no reset/email verification/rate limiting).

- Portable local auth is “shared secret”; anyone with the same email+password can impersonate. Good for LAN testing, not a production auth system.
- Tauri “auto-start server” is dev-only (depends on Node/npm and repo layout). Shipping needs a hosted server or an embedded native signaling server.

Context Storage

`PROJECT\_CONTEXT.md` is the editable source of truth. `PROJECT\_CONTEXT.pdf` is regenerated from it so long context can be stored/read outside the chat window.

Planning Docs

App-first architecture plan rewrite lives at `docs/WEBRTC\_SEND\_PIPE\_ARCH\_PLAN\_v2.md` and is exported to `docs/WEBRTC\_SEND\_PIPE\_ARCH\_PLAN\_v2.pdf`.